Supervyse OPF Redfish Porting Guide

Revision 1.0
12 2024

Copyright (c) 2024, All Rights Reserved. Insyde Software.

No part of this publication may be reproduced, transmitted, transcribed, stored in a retrieval system, or translated into any language or computer language, in any form, or by any means, electronic, mechanical, magnetic, optical, chemical, manual or otherwise, without the prior written permission of Insyde Software Corp.

Disclaimer

Insyde Software provides this document and the programs "as is" without warranty of any kind, either expressed or implied, including, but not limited to, the implied warranties of merchantability or fitness for a particular purpose.

This document could contain technical inaccuracies or typographical errors. Changes are periodically made to the information herein; these changes will be incorporated in future revisions of this document. Insyde Software is under no obligation to notify any person of the changes.

The following trademarks are used in this document:

Insyde and InsydeH2O are registered trademarks or trademarks of Insyde Software. All other trademarks or trade names are property of their respective holders.

Contents

1 Introduction 6
1.1 Overview 6
1.2 Related Information 6
1.3 Terms 6
2 Redfish Code Location And Architecture 7
2.1 Redfish Code Location 7
2.2 Redfish Code Architecture 7
3 Implement OEM Schema Flow 9
3.1 Define Resource URL And Resource JSON Response 10
3.1.1 HelloServiceCollection 10
3.1.2 HelloService 10
3.2 Define Resource Schema(CSDL) Based On JSON Response 11
3.2.1 HelloServiceCollection 11
3.2.1.1 Creating Fake Resource JSON 11
3.2.1.2 Generating Resource Schema 11
3.2.1.3 Check And Modify Resource Schema 14
3.2.2 HelloService 15
3.2.2.1 Creating Fake Resource JSON 15
3.2.2.2 Generating Resource Schema 16
3.2.2.3 Check And Modify Resource Schema 19
3.3 Define The Privilege Of Resource 21
3.3.1 Modify Privilege File 21
3.3.2 Use OpenBMC Script (Oberon) 22
3.4 Implement HTTP Method For Resource 23
3.4.1 HelloServiceCollection 23
3.4.2 HelloService 24
3.4.3 Register Resource For bmcweb 26
3.5 Compile Redfish 27
4 Validate Redfish 28
4.1 Validate Redfish Resource and HTTP Methods 28
4.1.1 HelloServiceCollection 28
4.1.1.1 GET Method 28
4.1.1.2 POST Method 28
4.1.2 HelloService 29
4.1.2.1 GET Method 29
4.1.2.2 PATCH Method 30
4.1.2.3 POST Method 30
4.1.2.4 DELETE Method 31
4.2 Validate Redfish Resources and Schemas 31

Revision History

Revision Number	Description	Author	Reviewer	Release Date
1.0	Initial release.	Ezra Lin	Victor Wu	12, 2024

1 Introduction

The document's purpose is to help users understand how to add a new schema or property to Redfish.

1.1 Overview

The document's purpose is to help users understand how to add a new schema or property to Redfish.

This document is based on **bmcweb-override** for explanation.

1.2 Related Information

The following publications and sources of information are referred to by this document or may be useful to you:

- *Redfish Specification, DMTF, 2024, https://www.dmtf.org/sites/default/files/standards/documents/DSP0266_1.21.0.pdf*
- *OData CSDL, OASIS, 2014, https://docs.oasis-open.org/odata/odata/v4.0/os/part3-csdl/odata-v4.0-os-part3-csdl.html*
- *Redfish CSDL Usage, DMTF, 2019, https://www.dmtf.org/sites/default/files/Redfish_School-Redfish_CSDL_Usage.pdf*
- *Redfish Introduction to CSDL, DMTF, 2019, https://www.dmtf.org/sites/default/files/Redfish_School-Introduction_to_CSDL.pdf*
- *Redfish-Service-Validator, DMTF, 2024, https://github.com/DMTF/Redfish-Service-Validator*
- *Redfish-Schema-Creator, DMTF, 2022, https://github.com/DMTF/Redfish-Schema-Creator*
- *Redfish Validator User Guide, 1.0, Insyde Software, 2024*

1.3 Terms

The following terms are used throughout this document:

Bmcweb-override

Bmcweb-override is a mechanism that allow users to register multiple URLs and implement resources easily in bmcweb

2 Redfish Code Location And Architecture

This section will introduce where the Redfish code is located and its architecture.

2.1 Redfish Code Location

- G4 of Intel-Bhs: meta-insyde/meta-common/meta-obmc/recipes-phosphor/interfaces/bmcweb/source
- G4 of ast2700: meta-insyde/meta-evb/meta-ast2700evb/recipes-phosphor/interfaces/bmcweb/source
- G4 of ast2750: meta-insyde/meta-evb/meta-ast2750evb/recipes-phosphor/interfaces/bmcweb/source
- G4 of Oberon: meta-insyde/meta-nvidia/recipes-phosphor/interfaces/bmcweb/source

2.2 Redfish Code Architecture

Here is Redfish Code Architecture.

The "insyde" folder is used to store OEM schema code. The OEM schema is registered by **Bmcweb-override** method.

The "redfish-core" folder is used to store the original schema code.

```
bmcweb/source/
├── AGGREGATION.md
├── builddir
├── CLIENTS.md
├── COMMON_ERRORS.md
├── config
├── DBUS_USAGE.md
├── DEVELOPING.md
├── event-listener
├── HEADERS.md
├── http
├── include
├── insyde
│   ├── include
│   ├── lib
│   └── utils
├── LICENSE
├── meson.build
├── meson_options.txt
├── mointer-HMC
```

```
├── OEM_SCHEMAS.md
├── OWNERS
├── README.md
├── REDFISH_CHECKLIST.md
├── redfish-core
│   ├── include
│   ├── lib
│   ├── src
│   └── ut
├── Redfish.md
├── run-ci
├── scripts
├── setup.cfg
├── src
├── static
├── subprojects
├── test
└── TESTING.md
```

3 Implement OEM Schema Flow

This section introduces how to implement OEM schema sector by sector.

1. Define Resource URL And Resource JSON Response.
2. Define Resource Schema(CSDL) Based On JSON Response.
3. Define The Privilege Of Resource.
4. Implement HTTP Method For Resource.
5. Register Resource For bmcweb.
6. Compile Redfish

This document will use HelloServiceCollection and HelloService as examples.

In HelloServiceCollection, we would like users with normal privileges to be able to get all of the HelloService resource schemas, while users with higher privileges can create HelloService instances.

Table 3-1 HelloServiceCollection Support Method and Privilege Table

	Login	ConfigureManager	ConfigureComponent	ConfigureUsers	ConfigureSelf
GET	V				
POST		V	V		

In HelloService, we would like users with normal privileges to be able to get HelloService info, while users with higher privileges can modify or remove HelloService, and also users can execute actions for HelloService.

Table 3-2 HelloService Support Method and Privilege Table

	Login	ConfigureManager	ConfigureComponent	ConfigureUsers	ConfigureSelf
GET	V				
POST		V			
PATCH			V		
DELETE			V		

For more information about Redfish privileges, please refer to the Redfish Specification.

3.1 Define Resource URL And Resource JSON Response

3.1.1 HelloServiceCollection

We want to access the "/redfish/v1/HelloService" URL to the HelloServiceCollection resource.

Here is the HelloServiceCollection json response.

```
{
   "@odata.id": "/redfish/v1/HelloService",
   "@odata.type": "#HelloServiceCollection.HelloServiceCollection",
   "Description": "HelloServiceCollection",
   "Members": [
      {
         "@odata.id": "/redfish/v1/HelloService/Hello_1"
      }
   ],
   "Members@odata.count": 1,
   "Name": "HelloService Collection"
}
```

3.1.2 HelloService

We want to access the "/redfish/v1/HelloService/<HelloServiceId>" URL to the HelloServiceCollection resource.

Here is the HelloService json response.

```
{
   "@odata.id": "/redfish/v1/HelloService/Hello_1",
   "@odata.type": "#HelloService.v1_0_0.HelloService",
   "Description": "HelloService",
   "IntProp": 123,
   "FloatProp": 1.111,
   "StrProp": "test",
   "BoolProp": true,
   "StrEnumProp": [
      "Mother",
      "Father"
   ],
   "Name": "HelloService",
   "Managers": {
      "@odata.id": "/redfish/v1/Managers/BMC"
   },
   "Actions": {
      "#HelloService.TestAction": {
         "target": "/redfish/v1/HelloService/Actions/HelloService.TestAction"
      }
   }
}
```

3.2 Define Resource Schema(CSDL) Based On JSON Response

The resource schema(aka. CSDL) is used to specify what properties exist in a resource. It is also used for Redfish-Service-Validator validating resources.

Before creating a resource schema, you must know the knowledge of CSDL; please refer to below.

- OData CSDL
- Redfish CSDL Usage
- Redfish Introduction to CSDL

This document will show you how to use Redfish-Schema-Creator to create a resource schema.

Note: Redfish-Schema-Creator can almost create schema; however, you should check and modify it necessarily.

Here are the create resource schema steps by Redfish-Schema-Creator.

1. Creating "fake" resource JSON for Redfish-Schema-Creator.
2. Generating the resource schema by Redfish-Schema-Creator.
3. Check and modify the resource schema.

3.2.1 HelloServiceCollection

3.2.1.1 Creating Fake Resource JSON

The collection response is the default payload; According to the previously defined HelloServiceCollection JSON response, we just only set the Member link to the HelloService resource.

This is content in HelloServiceCollection.json.

```
{
   "@odata.id": "/redfish/v1/HelloServiceCollection",
   "@odata.type": "#HelloServiceCollection.v1_0_0.HelloServiceCollection",
   "Description": "HelloServiceCollection",
   "Members": [
      {
         "@odata.id": "/redfish/v1/HelloService/Hello_1"
      }
   ],
    "Members!link": "HelloService",
   "Members@odata.count": 1,
   "Name": "ResourceCollection"
}
```

3.2.1.2 Generating Resource Schema

Use the Redfish-Schema-Creator to create resource schema(HelloServiceCollection.v1_0_0.xml).

```
python3 csdl_creator.py HelloServiceCollection.json
```

This is the content of HelloServiceCollection.v1_0_0.xml.

```
<!---->
<!--
################################################################
#################      -->
<!--# Redfish Schema:  HelloService.v1_0_0                    -->
<!--#                                                      -->
<!--# For a detailed change log, see the README file contained in the DSP8010 bundle,     -->
<!--# available at http://www.dmtf.org/standards/redfish                    -->
<!--# Copyright 2020 DMTF.                                  -->
<!--# For the full DMTF copyright policy, see http://www.dmtf.org/about/policies/copyright -->
<!--
################################################################
#################      -->
<!---->
<?xml version="1.0" ?>
<edmx:Edmx xmlns:edmx="http://docs.oasis-open.org/odata/ns/edmx" Version="4.0">
    <edmx:Reference Uri="http://redfish.dmtf.org/schemas/v1/Resource_v1.xml">
        <edmx:Include Namespace="Resource.v1_0_0"/>
    </edmx:Reference>
    <edmx:Reference Uri="http://docs.oasis-
open.org/odata/odata/v4.0/errata03/csd01/complete/vocabularies/Org.OData.Capabilities.V1.xml">
        <edmx:Include Namespace="Org.OData.Capabilities.V1" Alias="Capabilities"/>
    </edmx:Reference>
    <edmx:Reference Uri="http://docs.oasis-
open.org/odata/odata/v4.0/errata03/csd01/complete/vocabularies/Org.OData.Core.V1.xml">
        <edmx:Include Namespace="Org.OData.Core.V1" Alias="OData"/>
    </edmx:Reference>
    <edmx:Reference Uri="http://redfish.dmtf.org/schemas/v1/Manager_v1.xml">
        <edmx:Include Namespace="Manager"/>
    </edmx:Reference>
    <edmx:Reference Uri="http://redfish.dmtf.org/schemas/v1/RedfishExtensions_v1.xml">
        <edmx:Include Namespace="RedfishExtensions.v1_0_0" Alias="Redfish"/>
    </edmx:Reference>
    <edmx:DataServices>
        <Schema xmlns="http://docs.oasis-open.org/odata/ns/edm" Namespace="HelloService">
            <Annotation Term="Redfish.OwningEntity" String="TBD"/>
            <EntityType Name="HelloService" BaseType="Resource.v1_0_0.Resource"
Abstract="true">
                <Annotation Term="OData.Description" String="Parameters for HelloService."/>
                <Annotation Term="OData.LongDescription" String="Parameters for HelloService."/>
                <Annotation Term="Capabilities.InsertRestrictions">
                    <Record>
                        <PropertyValue Property="Insertable" Bool="false"/>
                    </Record>
                </Annotation>
                <Annotation Term="Capabilities.UpdateRestrictions">
                    <Record>
                        <PropertyValue Property="Updatable" Bool="false"/>
                    </Record>
                </Annotation>
                <Annotation Term="Capabilities.DeleteRestrictions">
                    <Record>
                        <PropertyValue Property="Deletable" Bool="false"/>
                    </Record>
```

```
                </Annotation>
            </EntityType>
        </Schema>
        <Schema xmlns="http://docs.oasis-open.org/odata/ns/edm"
Namespace="HelloService.v1_0_0">
            <Annotation Term="Redfish.OwningEntity" String="TBD"/>
            <Annotation Term="Redfish.Release" String="TBD"/>
            <EntityType Name="HelloService" BaseType="HelloService.HelloService">
                <Property Name="IntProp" Type="Edm.Int64" Nullable="false">
                    <Annotation Term="OData.Permissions"
EnumMember="OData.Permission/Read"/>
                    <Annotation Term="OData.Description" String="This is an integer type property.
Its value range is from 100 to 200."/>
                    <Annotation Term="OData.LongDescription" String="TBD"/>
                    <Annotation Term="Redfish.Required"/>
                </Property>
                <Property Name="FloatProp" Type="Edm.Decimal" Nullable="false">
                    <Annotation Term="OData.Permissions"
EnumMember="OData.Permission/Read"/>
                    <Annotation Term="OData.Description" String="This is a number type
property."/>
                    <Annotation Term="OData.LongDescription" String="TBD"/>
                </Property>
                <Property Name="StrProp" Type="Edm.String" Nullable="false">
                    <Annotation Term="OData.Permissions"
EnumMember="OData.Permission/Read"/>
                    <Annotation Term="OData.Description" String="This is a string type property."/>
                    <Annotation Term="OData.LongDescription" String="TBD"/>
                </Property>
                <Property Name="BoolProp" Type="Edm.Boolean" Nullable="false">
                    <Annotation Term="OData.Permissions"
EnumMember="OData.Permission/ReadWrite"/>
                    <Annotation Term="OData.Description" String="This is a boolean type
property."/>
                    <Annotation Term="OData.LongDescription" String="TBD"/>
                </Property>
                <Property Name="StrEnumProp" Type="HelloService.v1_0_0.StrEnumProp"
Nullable="false">
                    <Annotation Term="OData.Permissions"
EnumMember="OData.Permission/ReadWrite"/>
                    <Annotation Term="OData.Description" String="This is an array of string type
property. Allowed value: Mother, Father, Son, Daughter."/>
                    <Annotation Term="OData.LongDescription" String="TBD"/>
                    <Annotation Term="Redfish.Required"/>
                </Property>
                <NavigationProperty Name="Managers" Type="Manager.Manager" Nullable="false">
                    <Annotation Term="OData.Permissions"
EnumMember="OData.Permission/Read"/>
                    <Annotation Term="OData.Description" String="A link to Managers"/>
                    <Annotation Term="OData.LongDescription" String="This property shall be a link
to a resource collection of type Managers."/>
                </NavigationProperty>
                <Property Name="Actions" Type="HelloService.v1_0_0.Actions" Nullable="false">
                    <Annotation Term="OData.Permissions"
EnumMember="OData.Permission/Read"/>
                    <Annotation Term="OData.Description" String="TBD"/>
                    <Annotation Term="OData.LongDescription" String="TBD"/>
                </Property>
```

```
            </EntityType>
            <EnumType Name="StrEnumProp">
                <Member Name="Mother">
                    <Annotation Term="OData.Description" String="TBD"/>
                </Member>
                <Member Name="Father">
                    <Annotation Term="OData.Description" String="TBD"/>
                </Member>
                <Member Name="Son">
                    <Annotation Term="OData.Description" String="TBD"/>
                </Member>
                <Member Name="Daughter">
                    <Annotation Term="OData.Description" String="TBD"/>
                </Member>
            </EnumType>
            <ComplexType Name="TestAction" Nullable="false">
                <Annotation Term="OData.Description" String="TBD"/>
                <Annotation Term="OData.LongDescription" String="TBD"/>
                <Annotation Term="OData.AdditionalProperties" Bool="false"/>
                <Property Name="target" Type="Edm.String" Nullable="false">
                    <Annotation Term="OData.Permissions"
EnumMember="OData.Permission/Read"/>
                    <Annotation Term="OData.Description" String="TBD"/>
                    <Annotation Term="OData.LongDescription" String="TBD"/>
                </Property>
            </ComplexType>
            <ComplexType Name="Actions" Nullable="false">
                <Annotation Term="OData.Description" String="TBD"/>
                <Annotation Term="OData.LongDescription" String="TBD"/>
                <Annotation Term="OData.AdditionalProperties" Bool="false"/>
                <Property Name="TestAction" Type="HelloService.v1_0_0.TestAction"
Nullable="false">
                    <Annotation Term="OData.Permissions"
EnumMember="OData.Permission/Read"/>
                    <Annotation Term="OData.Description" String="TBD"/>
                    <Annotation Term="OData.LongDescription" String="TBD"/>
                </Property>
            </ComplexType>
        </Schema>
    </edmx:DataServices>
</edmx:Edmx>
```

3.2.1.3 Check And Modify Resource Schema

We hope the HelloServiceCollection supports POST method to create a HelloService instance; So we modify "<PropertyValue Property="Insertable" Bool="false"/>" to "<PropertyValue Property="Insertable" Bool="true"/>".

```
<Annotation Term="Capabilities.InsertRestrictions">
    <Record>
        <PropertyValue Property="Insertable" Bool="false"/>
    </Record>
</Annotation>
                    |
                    | Modify
                    |
```

```
                    \|/

<Annotation Term="Capabilities.InsertRestrictions">
    <Record>
        <PropertyValue Property="Insertable" Bool="true"/>
    </Record>
</Annotation>
```

We need to put HelloServiceCollection.xml into the "static/redfish/v1/schema" folder, after modifying HelloServiceCollection.xml.

3.2.2 HelloService

In HelloService, in addition to common properties(e.g. @odata.id, Name, and so on), we can also specify other types and attributes for properties.

Table 3.2-1 Property Type & Attribute Table

Property	Type	Attributes	Descriptrion
IntProp	integer	read-only (required)	This is an integer type property. Its value range is from 100 to 200.
FloatProp	number	read-only (null)	This is a number type property.
StrProp	string	read-only	This is a string type property.
BoolProp	boolean	read-write	This is a boolean type property.
StrEnumProp	string(enum)	read-write (required)	This is an array of string type property. Allowed value: Mother, Father, Son, Daughter.
Managers {	object		
@odata.id	string	read-only	Link to Manager resource schema.
}			

3.2.2.1 Creating Fake Resource JSON

Accroding to predefined HelloService json response and Property Type & Attribute, we create a fake resource json(HelloService.json) as below.

```
{
   "@odata.id": "/redfish/v1/HelloService/Hello_1",
   "@odata.type": "#HelloService.v1_0_0.HelloService",
   "Description": "HelloService",
   "IntProp": 123,
     "IntProp!description": "This is an integer type property. Its value range is from 100 to 200.",
     "IntProp!required": true,
   "FloatProp": 1.111,
     "FloatProp!description": "This is a number type property.",
   "StrProp": "test",
     "StrProp!description": "This is a string type property.",
   "BoolProp": true,
```

```
    "BoolProp!description": "This is a boolean type property.",
    "BoolProp!readonly": false,
  "StrEnumProp": "Mother | Father | Son | Daughter",
    "StrEnumProp!description": "This is an array of string type property. Allowed value: Mother,
Father, Son, Daughter.",
    "StrEnumProp!readonly": false,
    "StrEnumProp!required": true,
  "Name": "HelloService",
  "Managers": {
     "@odata.id": "/redfish/v1/Managers/BMC"
  },
    "Managers!link": "Manager",
  "Actions": {
     "TestAction": {
        "target": "/redfish/v1/HelloService/Actions/HelloService.TestAction"
     }
  }
}
```

3.2.2.2 Generating Resource Schema

Use the Redfish-Schema-Creator to create resource schema(HelloService.v1_0_0.xml).

```
python3 csdl_creator.py HelloService.json
```

This is the content of HelloService.v1_0_0.xml.

```
<!---->
<!--
##################################################################
#################      -->
<!--# Redfish Schema:  HelloService.v1_0_0                          -->
<!--#                                                          -->
<!--# For a detailed change log, see the README file contained in the DSP8010 bundle,     -->
<!--# available at http://www.dmtf.org/standards/redfish                          -->
<!--# Copyright 2020 DMTF.                                                  -->
<!--# For the full DMTF copyright policy, see http://www.dmtf.org/about/policies/copyright -->
<!--
##################################################################
#################      -->
<!---->
<?xml version="1.0" ?>
<edmx:Edmx xmlns:edmx="http://docs.oasis-open.org/odata/ns/edmx" Version="4.0">
    <edmx:Reference Uri="http://redfish.dmtf.org/schemas/v1/Resource_v1.xml">
        <edmx:Include Namespace="Resource.v1_0_0"/>
    </edmx:Reference>
    <edmx:Reference Uri="http://docs.oasis-
open.org/odata/odata/v4.0/errata03/csd01/complete/vocabularies/Org.OData.Capabilities.V1.xml">
        <edmx:Include Namespace="Org.OData.Capabilities.V1" Alias="Capabilities"/>
    </edmx:Reference>
    <edmx:Reference Uri="http://docs.oasis-
open.org/odata/odata/v4.0/errata03/csd01/complete/vocabularies/Org.OData.Core.V1.xml">
        <edmx:Include Namespace="Org.OData.Core.V1" Alias="OData"/>
    </edmx:Reference>
    <edmx:Reference Uri="http://redfish.dmtf.org/schemas/v1/Manager_v1.xml">
```

```
        <edmx:Include Namespace="Manager"/>
    </edmx:Reference>
    <edmx:Reference Uri="http://redfish.dmtf.org/schemas/v1/RedfishExtensions_v1.xml">
        <edmx:Include Namespace="RedfishExtensions.v1_0_0" Alias="Redfish"/>
    </edmx:Reference>
    <edmx:DataServices>
        <Schema xmlns="http://docs.oasis-open.org/odata/ns/edm" Namespace="HelloService">
            <Annotation Term="Redfish.OwningEntity" String="TBD"/>
            <EntityType Name="HelloService" BaseType="Resource.v1_0_0.Resource"
Abstract="true">
                <Annotation Term="OData.Description" String="Parameters for HelloService."/>
                <Annotation Term="OData.LongDescription" String="Parameters for HelloService."/>
                <Annotation Term="Capabilities.InsertRestrictions">
                    <Record>
                        <PropertyValue Property="Insertable" Bool="false"/>
                    </Record>
                </Annotation>
                <Annotation Term="Capabilities.UpdateRestrictions">
                    <Record>
                        <PropertyValue Property="Updatable" Bool="false"/>
                    </Record>
                </Annotation>
                <Annotation Term="Capabilities.DeleteRestrictions">
                    <Record>
                        <PropertyValue Property="Deletable" Bool="false"/>
                    </Record>
                </Annotation>
            </EntityType>
        </Schema>
        <Schema xmlns="http://docs.oasis-open.org/odata/ns/edm"
Namespace="HelloService.v1_0_0">
            <Annotation Term="Redfish.OwningEntity" String="TBD"/>
            <Annotation Term="Redfish.Release" String="TBD"/>
            <EntityType Name="HelloService" BaseType="HelloService.HelloService">
                <Property Name="IntProp" Type="Edm.Int64" Nullable="false">
                    <Annotation Term="OData.Permissions"
EnumMember="OData.Permission/Read"/>
                    <Annotation Term="OData.Description" String="This is an integer type property.
It value range is from 100 to 200."/>
                    <Annotation Term="OData.LongDescription" String="TBD"/>
                    <Annotation Term="Redfish.Required"/>
                </Property>
                <Property Name="FloatProp" Type="Edm.Decimal" Nullable="false">
                    <Annotation Term="OData.Permissions"
EnumMember="OData.Permission/Read"/>
                    <Annotation Term="OData.Description" String="This is a number type
property."/>
                    <Annotation Term="OData.LongDescription" String="TBD"/>
                </Property>
                <Property Name="StrProp" Type="Edm.String" Nullable="false">
                    <Annotation Term="OData.Permissions"
EnumMember="OData.Permission/Read"/>
                    <Annotation Term="OData.Description" String="This is a string type property."/>
                    <Annotation Term="OData.LongDescription" String="TBD"/>
                </Property>
                <Property Name="BoolProp" Type="Edm.Boolean" Nullable="false">
                    <Annotation Term="OData.Permissions"
EnumMember="OData.Permission/ReadWrite"/>
```

```
                    <Annotation Term="OData.Description" String="This is a boolean type
property."/>
                    <Annotation Term="OData.LongDescription" String="TBD"/>
                </Property>
                <Property Name="StrEnumProp" Type="HelloService.v1_0_0.StrEnumProp"
Nullable="false">
                    <Annotation Term="OData.Permissions"
EnumMember="OData.Permission/ReadWrite"/>
                    <Annotation Term="OData.Description" String="This is an array of string type
property. Allowed value: Mother, Father, Son, Daughter."/>
                    <Annotation Term="OData.LongDescription" String="TBD"/>
                    <Annotation Term="Redfish.Required"/>
                </Property>
                <NavigationProperty Name="Managers" Type="Manager.Manager" Nullable="false">
                    <Annotation Term="OData.Permissions"
EnumMember="OData.Permission/Read"/>
                    <Annotation Term="OData.Description" String="A link to Managers"/>
                    <Annotation Term="OData.LongDescription" String="This property shall be a link
to a resource collection of type Managers."/>
                </NavigationProperty>
                <Property Name="Actions" Type="HelloService.v1_0_0.Actions" Nullable="false">
                    <Annotation Term="OData.Permissions"
EnumMember="OData.Permission/Read"/>
                    <Annotation Term="OData.Description" String="TBD"/>
                    <Annotation Term="OData.LongDescription" String="TBD"/>
                </Property>
            </EntityType>
            <EnumType Name="StrEnumProp">
                <Member Name="Mother">
                    <Annotation Term="OData.Description" String="TBD"/>
                </Member>
                <Member Name="Father">
                    <Annotation Term="OData.Description" String="TBD"/>
                </Member>
                <Member Name="Son">
                    <Annotation Term="OData.Description" String="TBD"/>
                </Member>
                <Member Name="Daughter">
                    <Annotation Term="OData.Description" String="TBD"/>
                </Member>
            </EnumType>
            <ComplexType Name="TestAction" Nullable="false">
                <Annotation Term="OData.Description" String="TBD"/>
                <Annotation Term="OData.LongDescription" String="TBD"/>
                <Annotation Term="OData.AdditionalProperties" Bool="false"/>
                <Property Name="target" Type="Edm.String" Nullable="false">
                    <Annotation Term="OData.Permissions"
EnumMember="OData.Permission/Read"/>
                    <Annotation Term="OData.Description" String="TBD"/>
                    <Annotation Term="OData.LongDescription" String="TBD"/>
                </Property>
            </ComplexType>
            <ComplexType Name="Actions" Nullable="false">
                <Annotation Term="OData.Description" String="TBD"/>
                <Annotation Term="OData.LongDescription" String="TBD"/>
                <Annotation Term="OData.AdditionalProperties" Bool="false"/>
                <Property Name="TestAction" Type="HelloService.v1_0_0.TestAction"
Nullable="false">
```

```
                    <Annotation Term="OData.Permissions"
EnumMember="OData.Permission/Read"/>
                    <Annotation Term="OData.Description" String="TBD"/>
                    <Annotation Term="OData.LongDescription" String="TBD"/>
                </Property>
            </ComplexType>
        </Schema>
    </edmx:DataServices>
</edmx:Edmx>
```

3.2.2.3 Check And Modify Resource Schema

After generating the resource schema, we should fix three points in the resource schema.

1. Allow user to update and delete instances.
2. Allow the "FloatProp" property to be nullable.
3. Add action into resource schema.

We want to allow users to modify and delete instances, so the resource schema needs to be fixed as below.

```
<Annotation Term="Capabilities.UpdateRestrictions">
    <Record>
        <PropertyValue Property="Updatable" Bool="false"/>
    </Record>
</Annotation>
<Annotation Term="Capabilities.DeleteRestrictions">
    <Record>
        <PropertyValue Property="Deletable" Bool="false"/>
    </Record>
</Annotation>
                    |
                    | Modify
                    |
                   \|/

<Annotation Term="Capabilities.UpdateRestrictions">
    <Record>
        <PropertyValue Property="Updatable" Bool="true"/>
    </Record>
</Annotation>
<Annotation Term="Capabilities.DeleteRestrictions">
    <Record>
        <PropertyValue Property="Deletable" Bool="true"/>
    </Record>
</Annotation>
```

We want to allow "FloatProp" to be nullable, so the resource schema needs to be fixed as below.

```
<Property Name="FloatProp" Type="Edm.Decimal" Nullable="false">
    <Annotation Term="OData.Permissions" EnumMember="OData.Permission/Read"/>
    <Annotation Term="OData.Description" String="This is an number type property."/>
    <Annotation Term="OData.LongDescription" String="TBD"/>
</Property>
                    |
                    | Modify
                    |
                   \|/

<Property Name="FloatProp" Type="Edm.Decimal" Nullable="true">
    <Annotation Term="OData.Permissions" EnumMember="OData.Permission/Read"/>
    <Annotation Term="OData.Description" String="This is an number type property."/>
    <Annotation Term="OData.LongDescription" String="TBD"/>
</Property>
```

Resource schema lacks of TestAction, so we need to fix it as below.

```
<Schema xmlns="http://docs.oasis-open.org/odata/ns/edm" Namespace="HelloService">
    <Annotation Term="Redfish.OwningEntity" String="TBD"/>
    <EntityType Name="HelloService" BaseType="Resource.v1_0_0.Resource" Abstract="true">
        <Annotation Term="OData.Description" String="Parameters for HelloService."/>
        <Annotation Term="OData.LongDescription" String="Parameters for HelloService."/>
        ...< other infomation >...
    </EntityType>
</Schema>
                    |
                    | Modify
                    |
                   \|/

<Schema xmlns="http://docs.oasis-open.org/odata/ns/edm" Namespace="HelloService">
    <Annotation Term="Redfish.OwningEntity" String="TBD"/>
    <EntityType Name="HelloService" BaseType="Resource.v1_0_0.Resource" Abstract="true">
        <Annotation Term="OData.Description" String="Parameters for HelloService."/>
        <Annotation Term="OData.LongDescription" String="Parameters for HelloService."/>

        ...< other infomation >...

        <Action Name="TestAction" IsBound="true">
            <Annotation Term="OData.Description" String="This TestAction of HelloService"/>
            <Annotation Term="OData.LongDescription" String="TBD"/>
        </Action>
    </EntityType>
</Schema>
```

We need to put HelloService.xml into the "static/redfish/v1/schema" folder, after modifying HelloService.xml.

3.3 Define The Privilege Of Resource

In HelloServiceCollection, we would like users with the normal privileges to be able to get all of the HelloService resource schema, while users with higher privileges can create HelloService instances.

In HelloService, we would like users with normal privileges to be able to get HelloService info, while users with higher privileges can modify or remove HelloService, and also users can execute actions for HelloService.

Table 3.3-1 HelloServiceCollection Support Method and Privilege Table

	Login	ConfigureManager	ConfigureComponent	ConfigureUsers	ConfigureSelf
GET	V				
POST		V	V		

Table 3.3-2 HelloService Support Method and Privilege Table

	Login	ConfigureManager	ConfigureComponent	ConfigureUsers	ConfigureSelf
GET	V				
POST		V			
PATCH			V		
DELETE			V		

There are two method to define the privilege. One is to modify redfish-core/include/privileges.hpp, and the other is to use an OpenBMC supported script to define it (Only Supported on Oberon).

- Modify Privilege File
- Use OpenBMC Script (Oberon)

3.3.1 Modify Privilege File

Add the privileges for HelloServiceCollection and HelloService(following example) to redfish-core/include/privileges.hpp.

```
// In redfish-core/include/privileges.hpp.

// HelloServiceCollection
const static auto& getHelloServiceCollection = privilegeSetLogin
const static auto& postHelloServiceCollection = privilegeSetConfigureManager

// HelloService
const static auto& getHelloService = privilegeSetLogin
const static auto& postHelloService = privilegeSetConfigureManager
const static auto& patchHelloService = privilegeSetConfigureComponents
const static auto& deleteHelloService = privilegeSetConfigureComponents
```

3.3.2 Use OpenBMC Script (Oberon)

Add the privilege for HelloService and HelloServiceCollection(following example) to the "Mapping" array in insyde_privilege.json.

```
{
   "Entity": "HelloServiceCollection",
   "OperationMap": {
      "GET": [
         {
            "Privilege": [
               "Login"
            ]
         }
      ],
      "POST": [
         {
            "Privilege": [
               "ConfigureManager"
            ]
         }
      ]
   }
},
{
   "Entity": "HelloService",
   "OperationMap": {
      "GET": [
         {
            "Privilege": [
               "Login"
            ]
         }
      ],
      "POST": [
         {
            "Privilege": [
               "ConfigureManager"
            ]
         }
      ],
      "PATCH": [
         {
            "Privilege": [
               "ConfigureComponents"
            ]
         }
      ],
      "DELETE": [
         {
            "Privilege": [
               "ConfigureComponents"
            ]
         }
      ]
   }
}
```

After modifying insyde_privilege.json, you can execute scripts/parse_registries.py to generate privilege_registry.hpp.

```
python3 parse_registries.py
```

The privilege_registry.hpp final result:

```
// In redfish-core/include/privileges.hpp.

// HelloServiceCollection
const static auto& getHelloServiceCollection = privilegeSetLogin
const static auto& postHelloServiceCollection = privilegeSetConfigureManager

// HelloService
const static auto& getHelloService = privilegeSetLogin
const static auto& postHelloService = privilegeSetConfigureManager
const static auto& patchHelloService = privilegeSetConfigureComponents
const static auto& deleteHelloService = privilegeSetConfigureComponents
```

3.4 Implement HTTP Method For Resource

We can implement the HTTP method for resources on insyde/lib/insyde_hello_service.hpp.

3.4.1 HelloServiceCollection

We can implement the HelloServiceCollection resource based on the previously defined HTTP response, privileges, and HTTP method.

Note: It doesn't necessarily have to be implemented in insyde/lib/insyde_hello_service.hpp; it can be implemented elsewhere. This example uses insyde/lib/insyde_hello_service.hpp as a reference.

The following example is represented using pseudocode:

```
// In insyde/lib/insyde_hello_service.hpp.
#pragma once

#include "app.hpp"

namespace redfish
{

inline void handlePostHelloService(crow::App& app, const crow::Request& req,
                                   const std::shared_ptr<bmcweb::AsyncResp>& asyncResp)
{
   // According to HelloServcie Property Type and Attribute,
   // the BoolProp and StrEnumProp are writable,
   // so we can create a HelloService resource using them
   bool BoolProp;
   std::optional<std::vector> strEnumProp;

   if (!json_util::readJsonPatch(req, asyncResp->res, "BoolProp",
                                 BoolProp, "StrEnumProp", strEnumProp))
   {
      return;
   }
   // Create HelloService instance by StrEnumProp and BoolProp
```

```
    createHelloService(asyncResp, BoolProp, strEnumProp);
}

inline void handleGetHelloServiceCollection(crow::App& app, const crow::Request& req,
                                            const std::shared_ptr<bmcweb::AsyncResp>& asyncResp)
{
    // Add common properties to JSON response. (e.g. @odata.id, @odata.type, Name...)
    asyncResp->res.jsonValue["@odata.id"] = "/redfish/v1/HelloService"

    // Find all of the HelloService instances and add them under the object JSON in the following
format.
    // "Members": [
    //     {
    //         "@odata.id": "/redfish/v1/HelloService/Hello_1"
    //     }
    // ]
    asyncResp->res.jsonValue["Members"] = nlohmann::json::array();
    nlohmann::json& Members = asyncRes->res.jsonValue["Members"];
    for (HelloService : TotalHelloService)
    {
        nlohmann::json::object_t object;
        object["@odata.id"] = "/redfish/v1/HelloService/Hello_" + std::to_string(id);
        object.emplace_back(std::move(member));
        Members.emplace_back(object);
    }
}

inline void requestRoutesHelloServiceCollection(App& app)
{
    // Set the URL of the HelloServiceCollection resource to /redfish/v1/HelloService,
    // specify the HTTP method as GET, and assign the privilege as Login.
    BMCWEB_ROUTE(app, "/redfish/v1/HelloService/")
        .privileges(redfish::privileges::getHelloServiceCollection)
        .methods(boost::beast::http::verb::get)(
            std::bind_front(handleGetHelloServiceCollection, std::ref(app)));

    // Set the URL of the HelloServiceCollection resource to /redfish/v1/HelloService,
    // specify the HTTP method as POST, and assign the privilege as ConfigureManager.
    BMCWEB_ROUTE(app, "/redfish/v1/HelloService/")
        .privileges(redfish::privileges::postHelloServiceCollection)
        .methods(boost::beast::http::verb::post)(
            std::bind_front(handlePostHelloService, std::ref(app)));
}

} // namespace redfish
```

3.4.2 HelloService

We can implement the HelloService resource based on the previously defined HTTP response, privileges, and HTTP method.

The following example is represented using pseudocode:

```
// In insyde/lib/insyde_hello_service.hpp.
#pragma once

#include "app.hpp"
```

```
namespace redfish
{

inline void handleHelloServiceAction(crow::App& app, const crow::Request& req,
                                     const std::shared_ptr<bmcweb::AsyncResp>& asyncResp)
{
    // Execute HelloService action.
    executeHelloServiceAction(asyncResp);
}

inline void handleDeleteHelloService(crow::App& app, const crow::Request& req,
                                     const std::shared_ptr<bmcweb::AsyncResp>& asyncResp,
                                     const std::string& Id)
{
    // Remove the specific HelloService instance.
    removeHelloServiceById(asyncResp, Id);
}

inline void handlePatchHelloService(crow::App& app, const crow::Request& req,
                                    const std::shared_ptr<bmcweb::AsyncResp>& asyncResp,
                                    const std::string& Id)
{
    // According to HelloServcie Property Type and Attribute,
    // the BoolProp and StrEnumProp are writable,
    // so we can update HelloService resources using them
    bool BoolProp;
    std::optional<std::vector> strEnumProp;

    if (!json_util::readJsonPatch(req, asyncResp->res, "BoolProp",
                                  BoolProp, "StrEnumProp", strEnumProp))
    {
        return;
    }
    // Update HelloService instance by StrEnumProp and BoolProp
    updateHelloService(asyncResp, BoolProp, strEnumProp);
}

inline void handleGetHelloService(crow::App& app, const crow::Request& req,
                                  const std::shared_ptr<bmcweb::AsyncResp>& asyncResp,
                                  const std::string& Id)
{
    // Add common properties to JSON response. (e.g. @odata.id, @odata.type, Name...)
    asyncResp->res.jsonValue["@odata.id"] = "/redfish/v1/HelloService/Hello_" + Id

    // Get HelloService info and add it to the JSON response. (e.g. IntProp, FloatProp and BoolProp...)
    getHelloServiceInfo(asyncResp);
}

inline void requestRoutesHelloService(App& app)
{
    // Set the URL of the HelloService resource to /redfish/v1/HelloService/<HelloServiceId>,
    // specify the HTTP method as GET, and assign the privilege as Login.
    BMCWEB_ROUTE(app, "/redfish/v1/HelloService/<str>")
        .privileges(redfish::privileges::getHelloService)
        .methods(boost::beast::http::verb::get)(
            std::bind_front(handleGetHelloService, std::ref(app)));
```

```
    // Set the URL of the HelloService resource to /redfish/v1/HelloService/<HelloServiceId>,
    // specify the HTTP method as PATCH, and assign the privilege as ConfigureComponents.
    BMCWEB_ROUTE(app, "/redfish/v1/HelloService/<str>")
        .privileges(redfish::privileges::patchHelloService)
        .methods(boost::beast::http::verb::patch)(
            std::bind_front(handlePatchHelloService, std::ref(app)));

    // Set the URL of the HelloService resource to /redfish/v1/HelloService/<HelloServiceId>,
    // specify the HTTP method as DELETE, and assign the privilege as ConfigureComponents.
    BMCWEB_ROUTE(app, "/redfish/v1/HelloService/<str>")
        .privileges(redfish::privileges::deleteHelloService)
        .methods(boost::beast::http::verb::delete_)(
            std::bind_front(handleDeleteHelloService, std::ref(app)));

    // Set the URL of the HelloService action to
/redfish/v1/HelloService/Actions/HelloService.TestAction,
    // specify the HTTP method as POST, and assign the privilege as ConfigureManager.
    BMCWEB_ROUTE(app, "/redfish/v1/HelloService/Actions/HelloService.TestAction/")
        .privileges(redfish::privileges::postHelloService)
        .methods(boost::beast::http::verb::post)(
            std::bind_front(handleHelloServiceAction, std::ref(app)));
}

} // namespace redfish
```

3.4.3 Register Resource For bmcweb

We can register resources for bmcweb after implementing resources.

There are two functions(e.g. requestRoutesHelloService and requestRoutesHelloServiceCollection) in insyde/lib/insyde_hello_service.hpp that need to be registered in redfish/include/insyde_redfish.hpp.

Note: It doesn’t necessarily have to be registered in redfish/include/insyde_redfish.hpp; it can be implemented elsewhere. This example uses insyde/include/insyde_redfish.hpp as a reference.

The following example is represented using pseudocode:

```
// In insyde/include/insyde_redfish.hpp.
#pragma once

#include "../lib/insyde_hello_service.hpp"

namespace redfish
{

inline void requestRoutesInsydeRedfish(App& app)
{
    // Register HelloServiceCollection and HelloService resource for Insdye Redfish
    requestRoutesHelloServiceCollection(app);
    requestRoutesHelloService(app);
}

} // namespace redfish
```

Finally, we need to register Insyde Redfish for bmcweb; We should add the requestRoutesInsydeRedfish function in redfish-core/include/redfish.hpp.

The following example is represented using pseudocode:

```
// In redfish-core/include/redfish.hpp.
#pragma once

#include "insyde/include/insyde_redfish.hpp"

namespace redfish
{
/*
 * @brief Top level class installing and providing Redfish services
 */
class RedfishService
{
  public:
    /*
     * @brief Redfish service constructor
     *
     * Loads Redfish configuration and installs schema resources
     *
     * @param[in] app   Crow app on which Redfish will initialize
     */
    explicit RedfishService(App& app)
    {
        // Register Insyde Redfish (HelloServcie and HelloServiceCollection) for bmcweb
        requestRoutesInsydeRedfish(app);
        // Note, this must be the last route registered
        requestRoutesRedfish(app);
    }
};


} // namespace redfish
```

3.5 Compile Redfish

After you modify the redfish code, you can compile redfish.

```
bitbake bmcweb
```

4 Validate Redfish

We have compiled the Redfish and can now start validating HelloService and HelloServiceCollection schemas and resources.

4.1 Validate Redfish Resource and HTTP Methods

This section will use curl to validate the Redfish resources' HTTP method and response.

4.1.1 HelloServiceCollection

We implemented GET and POST methods in the HelloServiceCollection so that we can validate both functionalities.

4.1.1.1 GET Method

Here are the curl and JSON response examples for validating the HelloServiceCollection with the GET Method.

```
curl -k -u <BMC UserName>:<BMC Password> -X GET https://<BMC Server
Address>/redfish/v1/HelloService

{
   "@odata.id": "/redfish/v1/HelloService",
   "@odata.type": "#HelloServiceCollection.HelloServiceCollection",
   "Description": "HelloServiceCollection",
   "Members": [
      {
         "@odata.id": "/redfish/v1/HelloService/Hello_1"
      },
      {
         "@odata.id": "/redfish/v1/HelloService/Hello_2"
      }
   ],
   "Members@odata.count": 2,
   "Name": "HelloService Collection"
}
```

4.1.1.2 POST Method

Here is the curl example for validating the HelloServiceCollection with the POST Method to create the HelleServie instance with BoolProp and StrEnmProp.

```
curl -k -u <BMC UserName>:<BMC Password> -X POST -H "Content-Type: application/json"
https://<BMC Server Address>/redfish/v1/HelloService -d '{"BoolProp": false,
"StrEnumProp":["Father"]}'
```

After creating the HelloService instance, we can access HelloServiceCollection and HelloService to validate it. Here are the HelloServiceCollection and HelloService(Hello_3) JSON response examples.

```
{
   "@odata.id": "/redfish/v1/HelloService",
```

```
  "@odata.type": "#HelloServiceCollection.HelloServiceCollection",
  "Description": "HelloServiceCollection",
  "Members": [
    {
      "@odata.id": "/redfish/v1/HelloService/Hello_1"
    },
    {
      "@odata.id": "/redfish/v1/HelloService/Hello_2"
    },
    {
      "@odata.id": "/redfish/v1/HelloService/Hello_3"
    }
  ],
  "Members@odata.count": 3,
  "Name": "HelloService Collection"
}

{
  "@odata.id": "/redfish/v1/HelloService/Hello_3",
  "@odata.type": "#HelloService.v1_0_0.HelloService",
  "Description": "HelloService",
  "IntProp": 123,
  "FloatProp": 1.111,
  "StrProp": "string",
  "BoolProp": false,
  "StrEnumProp": [
    "Father"
  ],
  "Name": "HelloService",
  "Managers": {
    "@odata.id": "/redfish/v1/Managers/BMC"
  },
  "Actions": {
    "#HelloService.TestAction": {
      "target": "/redfish/v1/HelloService/Actions/HelloService.TestAction"
    }
  }
}
```

4.1.2 HelloService

We implemented GET, POST, PATCH, and DELETE methods in the HelloService so that we can validate these functionalities.

4.1.2.1 GET Method

Here are the curl and JSON response examples for validating the HelloService with the GET Method.

```
curl -k -u <BMC UserName>:<BMC Password> -X GET https://<BMC Server
Address>/redfish/v1/HelloService/Hello_1

{
  "@odata.id": "/redfish/v1/HelloService/Hello_1",
  "@odata.type": "#HelloService.v1_0_0.HelloService",
  "Description": "HelloService",
  "IntProp": 1223,
```

```
  "FloatProp": 1.111,
  "StrProp": "string",
  "BoolProp": true,
  "StrEnumProp": [
    "Mother"
  ],
  "Name": "HelloService",
  "Managers": {
    "@odata.id": "/redfish/v1/Managers/BMC"
  },
  "Actions": {
    "#HelloService.TestAction": {
      "target": "/redfish/v1/HelloService/Actions/HelloService.TestAction"
    }
  }
}
```

4.1.2.2 PATCH Method

Here is the curl example for validating the HelloService with the PATCH Method to update the HelleServie(Hello_1) instance with BoolProp.

```
curl -k -u <BMC UserName>:<BMC Password> -X PATCH -H "Content-Type: application/json"
https://<BMC Server Address>/redfish/v1/HelloService/Hello_1 -d '{"BoolProp": true}'
```

After updateing the HelloService, we can access HelloService to validate it. Here is the HelloService(Hello_1) JSON response example.

```
{
  "@odata.id": "/redfish/v1/HelloService/Hello_1",
  "@odata.type": "#HelloService.v1_0_0.HelloService",
  "Description": "HelloService",
  "IntProp": 1223,
  "FloatProp": 1.111,
  "StrProp": "string",
  "BoolProp": true,
  "StrEnumProp": [
    "Mother"
  ],
  "Name": "HelloService",
  "Managers": {
    "@odata.id": "/redfish/v1/Managers/BMC"
  },
  "Actions": {
    "#HelloService.TestAction": {
      "target": "/redfish/v1/HelloService/Actions/HelloService.TestAction"
    }
  }
}
```

4.1.2.3 POST Method

Here is the curl example for validating the HelloService action.

```
curl -k -u <BMC UserName>:<BMC Password> -X POST https://<BMC Server
Address>/redfish/v1/HelloService/Actions/HelloService.TestAction
```

4.1.2.4 DELETE Method

Here is the curl example for validating the HelloService with the DELETE Method to delete the HelleServie(Hello_3) instance.

```
curl -k -u <BMC UserName>:<BMC Password> -X DELETE https://<BMC Server Address>/redfish/v1/HelloService/Hello_3
```

After deleting the HelloService instance, we can access HelloServiceCollection to validate it. Here are the HelloServiceCollection JSON response examples.


```
{
   "@odata.id": "/redfish/v1/HelloService",
   "@odata.type": "#HelloServiceCollection.HelloServiceCollection",
   "Description": "HelloServiceCollection",
   "Members": [
      {
         "@odata.id": "/redfish/v1/HelloService/Hello_1"
      },
      {
         "@odata.id": "/redfish/v1/HelloService/Hello_2"
      },
   ],
   "Members@odata.count": 2,
   "Name": "HelloService Collection"
}
```


4.2 Validate Redfish Resources and Schemas

We can use the Redfish Service Validator to validate Redfish resources and schemas.

Please refer to the Redfish Validator User Guide to configure the Redfish Service Validator. Remember to place HelloService.xml and HelloServiceCollection.xml in the folder specified by the Redfish Service Validator(schema_directory).

Validate HelloServiceCollection resource and schema:

```
python3 RedfishServiceValidator.py -c ./config.ini --payload Single /redfish/v1/HelloService
```

or

```
python3 RedfishServiceValidator.py -r <Server IP> -u <username> -p <password> --schema_directory </your/schema.xml/folder> --payload Single /redfish/v1/HelloService
```

Validate HelloServiceCollection resource and schema:

```
python3 RedfishServiceValidator.py -c ./config.ini --payload Single /redfish/v1/HelloService/Hello_1
```

or

```
python3 RedfishServiceValidator.py -r <Server IP> -u <username> -p <password> --schema_directory </your/schema.xml/folder> --payload Single /redfish/v1/HelloService/Hello_1
```